

Criação e uso de funções

Objetivo da Aula

- Entender o que é uma função e por que usar
 - Criar funções com e sem parâmetros
 - Utilizar `return` para devolver valores
 - Compreender escopo de variáveis (local vs global)
 - Combinar funções para organizar código de testes
-

1. O que é uma Função

Uma função é um bloco de código reutilizável que recebe um nome e pode ser chamado quantas vezes for necessário. Em vez de copiar e colar o mesmo trecho em vários lugares, você define uma função e a chama onde precisar.

Pense em uma função como uma **receita**: você escreve uma vez e segue sempre que precisar do resultado. A receita de "gerar relatório" ou "validar campo" é escrita uma vez e executada quantas vezes for necessário.

A sintaxe básica usa a palavra-chave `def`, seguida do nome da função, parênteses e dois pontos:

```
def exibir_separador():  
    print("-" * 40)  
  
# Chamando a função  
exibir_separador()  
print("Relatório de Testes")  
exibir_separador()
```

Dica de QA: Sem funções, cada teste repetiria código de setup, validação e teardown. Com funções, você escreve uma vez e reutiliza em toda a suite.

2. Funções com Parâmetros

Parâmetros são variáveis que a função recebe entre parênteses. Eles tornam a função flexível, permitindo que ela trabalhe com dados diferentes a cada chamada.

```
def exibir_status_teste(nome_teste, status):  
    print(f"Teste: {nome_teste} | Status: {status}")  
  
exibir_status_teste("Login com credenciais válidas", "PASSOU")  
exibir_status_teste("Login com senha incorreta", "PASSOU")  
exibir_status_teste("Login sem preencher email", "FALHOU")
```

Funções podem usar os parâmetros em qualquer lógica interna:

```
def classificar_status_codigo(codigo):  
    if 200 <= codigo < 300:  
        print(f"Status {codigo}: Sucesso")  
    elif 400 <= codigo < 500:  
        print(f"Status {codigo}: Erro do cliente")  
    elif 500 <= codigo < 600:  
        print(f"Status {codigo}: Erro do servidor")
```

Dica de QA: Em testes de API, uma função que classifica status codes evita repetir a mesma lógica de if/elif em cada teste.

3. Funções com Retorno

A palavra-chave `return` faz a função devolver um valor para quem a chamou. Esse valor pode ser armazenado em uma variável e usado em qualquer lugar do código.

```
def calcular_taxa_sucesso(total, aprovados):  
    if total == 0:  
        return 0  
    return (aprovados / total) * 100
```

```
taxa = calcular_taxa_sucesso(10, 8)
print(f"Taxa de sucesso: {taxa}%") # 80.0%
```

Imagine a diferença: `print` é como **falar em voz alta** (mostra na tela e pronto). `return` é como **entregar um documento** (devolve o valor para alguém usar depois).

Exemplo prático com geração de dados de teste:

```
def gerar_email_teste(nome, dominio="qatest.com"):
    return f"{nome.lower().replace(' ', '.')}@{dominio}"

email = gerar_email_teste("João Silva")
# resultado: joao.silva@qatest.com
```

Dica de QA: Funções que retornam valores são a base das assertions em testes. Você chama a função, pega o retorno e compara com o esperado.

4. Funções sem Retorno (None)

Quando uma função não possui `return`, ela retorna `None` automaticamente. Funções assim são usadas para executar ações (imprimir, salvar, logar) sem precisar devolver um valor.

```
def logar_acao(acao):
    print(f"[LOG] Ação executada: {acao}")

resultado = logar_acao("Clicou no botão Salvar")
print(resultado) # None
```

O `return` sem valor também pode ser usado para encerrar a função antes do fim:

```
def validar_campo_obrigatorio(campo, valor):
    if not valor:
        print(f"[ERRO] Campo '{campo}' está vazio!")
```

```
    return # Encerra aqui, não executa o resto
print(f"[OK] Campo '{campo}' preenchido: {valor}")
```

Dica de QA: Cuidado ao tentar usar o retorno de uma função que não retorna nada. O valor será `None` e pode causar bugs silenciosos nos seus testes.

5. Escopo de Variáveis

Variáveis criadas dentro de uma função existem apenas dentro dela. Isso se chama **escopo local**. Variáveis criadas fora de funções têm **escopo global** e podem ser lidas (mas não modificadas) dentro de funções.

```
ambiente_global = "produção"

def configurar_teste():
    ambiente_local = "homologação"
    print(ambiente_local) # Funciona
    print(ambiente_global) # Funciona (leitura)

configurar_teste()
# print(ambiente_local) # ERRO: variável não existe aqui
```

Variáveis com o mesmo nome em escopos diferentes são independentes:

```
url_base = "https://api.producao.com"

def obter_url_teste():
    url_base = "https://api.homologacao.com" # Local, não
    altera a global
    print(url_base) # homologacao

obter_url_teste()
print(url_base) # producao (inalterada)
```

Visualização do escopo:

ESCOPO GLOBAL

```
ambiente_global = "produção"
url_base = "api.producao.com"
```

ESCOPO LOCAL (função)

```
ambiente_local = "hml"
```

Pode **LER** globais ✓

Não **ALTERA** globais ✗

Dica de QA: Escopo protege seus testes. Se cada função tem suas próprias variáveis, um teste não interfere no outro acidentalmente.

6. Chamando Funções dentro de Funções

Funções podem chamar outras funções. Isso permite construir funcionalidades complexas a partir de peças simples e reutilizáveis.

```
def formatar_resultado(nome, status):
    icone = "✓" if status == "PASSOU" else "✗"
    return f"{icone} {nome}: {status}"

def gerar_relatorio_suite(nome_suite, resultados):
    print(f"Suite: {nome_suite}")
    print("=" * 40)
    for teste, status in resultados:
        linha = formatar_resultado(teste, status) # Chama
        # outra função
        print(f" {linha}")

    total = len(resultados)
    aprovados = sum(1 for _, s in resultados if s == "PASSO
U")
    taxa = calcular_taxa_sucesso(total, aprovados) # Reuti
    # liza
    print(f" Taxa de sucesso: {taxa}%")
```

```
# Chamando com dados reais de teste
testes_login = [
    ("Login válido", "PASSOU"),
    ("Login inválido", "PASSOU"),
    ("Login sem senha", "FALHOU"),
    ("Login bloqueado", "PASSOU"),
]

testes_cadastro = [
    ("Cadastro completo", "PASSOU"),
    ("Cadastro sem email", "PASSOU"),
    ("Cadastro duplicado", "FALHOU"),
]

gerar_relatorio_suite("Login", testes_login)
gerar_relatorio_suite("Cadastro", testes_cadastro)
```

Dica de QA: Essa composição é a base dos frameworks de teste. Uma função de setup chama funções de configuração, que chamam funções de conexão. Cada uma faz uma coisa e todas se combinam.

7. Boas Práticas ao Criar Funções

Prática	Exemplo Bom	Exemplo Ruim
Nome descritivo	<code>verificar_status_code</code>	<code>func1</code>
Responsabilidade única	Função que só extrai domínio	Função que extrai, valida e envia email
Tamanho reduzido	5 a 20 linhas	100+ linhas fazendo tudo
Retorno previsível	Sempre retorna <code>bool</code>	Às vezes retorna <code>str</code> , às vezes <code>None</code>

```
# Bom: cada função faz UMA coisa
def extrair_dominio(email):
    return email.split("@")[1]

def validar_dominio_permitido(email, dominios_permitidos):
```

```
dominio = extrair_dominio(email)
return dominio in dominios_permitidos
```

Dica de QA: Se sua função faz muitas coisas, quebre em funções menores. Funções pequenas são mais fáceis de testar, debugar e reutilizar.

Aplicações Práticas em QA

Funções aparecem em praticamente toda automação de testes:

Cenário	Função	Benefício
Geração de massa de teste	<code>gerar_email_teste(nome)</code>	Dados únicos a cada execução
Validação de respostas de API	<code>verificar_status_code(codigo)</code>	Lógica centralizada e reutilizável
Relatórios de execução	<code>gerar_relatorio_suite(nome, resultados)</code>	Padronização da saída
Setup e teardown de testes	<code>preparar_ambiente()</code> / <code>limpar_ambiente()</code>	Código organizado e manutenível
Validação de campos	<code>validar_campo_obrigatorio(campo, valor)</code>	Reuso em múltiplos formulários

Tabela Resumo

Conceito	Sintaxe	Quando Usar
Função simples	<code>def nome():</code>	Ação repetitiva sem dados externos
Com parâmetros	<code>def nome(param1, param2):</code>	Quando a função precisa de dados para trabalhar
Com retorno	<code>return valor</code>	Quando o resultado será usado depois
Sem retorno	Sem <code>return</code> (retorna <code>None</code>)	Ações como logar, imprimir, salvar
Escopo local	Variável dentro da função	Isolamento entre funções/testes
Composição	Função chama outra função	Organizar código complexo em partes simples

Na próxima aula, vamos explorar **parâmetros e argumentos** em profundidade: valores padrão, argumentos posicionais e nomeados, e como tornar suas funções ainda mais flexíveis e poderosas.